```python
from openpyxl import load_workbook

file = r"C:\Users\s4ty4m\Desktop\daily life\12612391.xlsx"
workbook = load_workbook(file, data_only=True)
sheet = workbook["Daily Log"]
print("Workbook loaded successfully")
print("Sheet name:", sheet.title)
```

```
Workbook loaded successfully
Sheet name: Daily Log
```

```python
try:
    workbook = load_workbook(
        file,
        data_only=True
    )
    sheet = workbook["Daily Log"]
    print("Excel file loaded successfully.")
    print("Worksheet:", sheet.title)
except FileNotFoundError:
    print("Error: Excel file was not found.")
    print("Make sure 12612391.xlsx is in the same folder as this
notebook.")
except KeyError:
    print("Error: Daily Log worksheet was not found.")
except Exception as error:
    print("Unexpected error:", error)
```

```
Excel file loaded successfully.
Worksheet: Daily Log
```

```python
print("Excel headings:")

for cell in sheet[5]:
    print(cell.value)
```

```
Excel headings:
Date
Sleep (min)
Fitness (min)
Study (min)
Coding (min)
Class (min)
Classes Attended
Other Activities (min)
Total Tracked (min)
Free/Unaccounted (min)
Day's Feeling
Satisfaction Level
Energy Level
```

```
Notes
None
```

```
records = []
errors = []

print("Data structures created.")
```

```
Data structures created.
```

```
from datetime import date

start_date = date(2026, 8, 17)
end_date = date(2026, 9, 21)

filtered_records = []

for record in records:

    record_date = record["date"]

    if hasattr(record_date, "date"):
        record_date = record_date.date()

    if start_date <= record_date <= end_date:
        filtered_records.append(record)

records = filtered_records

print("Project period:", start_date, "to", end_date)
print("Valid records in project period:", len(records))
```

```
Project period: 2026-08-17 to 2026-09-21
Valid records in project period: 0
```

```
def convert_rating_to_number(rating):

    if rating is None:
        return None

    rating = str(rating).strip().lower()

    # Day's Feeling
    if rating == "excellent":
        return 5

    elif rating == "good":
        return 4

    elif rating == "neutral":
        return 3
```

```
    elif rating == "low":
        return 2

    elif rating == "stressed":
        return 1

    # Satisfaction
    elif rating == "very satisfied":
        return 5

    elif rating == "satisfied":
        return 4

    elif rating == "unsatisfied":
        return 2

    elif rating == "very unsatisfied":
        return 1

    # Energy
    elif rating == "high":
        return 5

    elif rating == "medium":
        return 3

    elif rating == "low":
        return 1

    return None

def validate_minutes(value, name, row_number):

    try:
        value = float(value)

    except (ValueError, TypeError):

        return None, (
            "Row " + str(row_number) +
            ": Invalid " + name
        )

    if value < 0 or value > 1440:

        return None, (
            "Row " + str(row_number) +
            ": " + name +
            " must be between 0 and 1440 minutes"
        )
```

```
    return value, None

for row_number, row in enumerate(
    sheet.iter_rows(min_row=6, values_only=True),
    start=6
):

    try:
        date = row[0]

        sleep_minutes = row[1]
        fitness_minutes = row[2]
        study_minutes = row[3]
        coding_minutes = row[4]
        class_minutes = row[5]

        classes_attended = row[6]

        other_minutes = row[7]

        total_tracked = row[8]
        free_minutes = row[9]

        feeling = row[10]
        satisfaction = row[11]
        energy = row[12]

        notes = row[13]
        if date is None:
            continue
        sleep_minutes, error = validate_minutes(
            sleep_minutes,
            "Sleep",
            row_number
        )

        if error:
            errors.append(error)
            continue


        fitness_minutes, error = validate_minutes(
            fitness_minutes,
            "Fitness",
            row_number
        )

        if error:
            errors.append(error)
            continue
```

```
        study_minutes, error = validate_minutes(
            study_minutes,
            "Study",
            row_number
        )

        if error:
            errors.append(error)
            continue


        coding_minutes, error = validate_minutes(
            coding_minutes,
            "Coding",
            row_number
        )

        if error:
            errors.append(error)
            continue


        class_minutes, error = validate_minutes(
            class_minutes,
            "Class",
            row_number
        )

        if error:
            errors.append(error)
            continue


        other_minutes, error = validate_minutes(
            other_minutes,
            "Other Activities",
            row_number
        )

        if error:
            errors.append(error)
            continue
        try:
            classes_attended = float(
                classes_attended
            )

            if classes_attended < 0:
```

```
            raise ValueError

    except (ValueError, TypeError):

        errors.append(
            "Row " + str(row_number) +
            ": Invalid Classes Attended"
        )

        continue
    calculated_total = (
        sleep_minutes +
        fitness_minutes +
        study_minutes +
        coding_minutes +
        class_minutes +
        other_minutes
    )
    if calculated_total > 1440:

        errors.append(
            "Row " + str(row_number) +
            ": Total activity time exceeds 1440 minutes"
        )

        continue
    calculated_free = (
        1440 -
        calculated_total
    )

    if convert_rating_to_number(feeling) is None:

        errors.append(
            "Row " + str(row_number) +
            ": Invalid Day's Feeling"
        )

        continue


    if convert_rating_to_number(satisfaction) is None:

        errors.append(
            "Row " + str(row_number) +
            ": Invalid Satisfaction Level"
        )

        continue
```

```
        if convert_rating_to_number(energy) is None:

            errors.append(
                "Row " + str(row_number) +
                ": Invalid Energy Level"
            )

            continue
        day_data = {

            "date": date,

            "sleep_minutes": sleep_minutes,

            "fitness_minutes": fitness_minutes,

            "study_minutes": study_minutes,

            "coding_minutes": coding_minutes,

            "class_minutes": class_minutes,

            "classes_attended": classes_attended,

            "other_minutes": other_minutes,

            "total_tracked": calculated_total,

            "free_minutes": calculated_free,

            "feeling": feeling,

            "satisfaction": satisfaction,

            "energy": energy,

            "notes": notes
        }


        records.append(day_data)


    except Exception as error:

        errors.append(
            "Row " + str(row_number) +
            ": " + str(error)
        )
```

```
print("Valid records:", len(records))
print("Errors:", len(errors))

if len(errors) == 0:

    print("No errors found.")

else:

    print("\nErrors found:")

    for error in errors:
        print(error)
```

```
Valid records: 39
Errors: 0
No errors found.
```

```
if len(records) > 0:

    print("First valid record:")
    print(records[0])
```

```
First valid record:
{'date': datetime.datetime(2026, 8, 17, 0, 0), 'sleep_minutes': 480.0,
'fitness_minutes': 30.0, 'study_minutes': 180.0, 'coding_minutes':
60.0, 'class_minutes': 300.0, 'classes_attended': 6.0,
'other_minutes': 60.0, 'total_tracked': 1110.0, 'free_minutes': 330.0,
'feeling': 'Good', 'satisfaction': 'Neutral', 'energy': 'Medium',
'notes': 'college classes and coding practice and felt a little tired
after class'}
```

```
def find_average(records, key_name):

    total = 0
    count = 0

    for record in records:

        value = record[key_name]

        if value is not None:

            total = total + value
            count = count + 1

    if count == 0:
        return 0

    return total / count
```

```
average_sleep = find_average(
    records,
    "sleep_minutes"
)

average_fitness = find_average(
    records,
    "fitness_minutes"
)

average_study = find_average(
    records,
    "study_minutes"
)

average_coding = find_average(
    records,
    "coding_minutes"
)

average_class = find_average(
    records,
    "class_minutes"
)

average_other = find_average(
    records,
    "other_minutes"
)

average_total = find_average(
    records,
    "total_tracked"
)

average_free = find_average(
    records,
    "free_minutes"
)


print("Average Sleep:", round(average_sleep, 2))
print("Average Fitness:", round(average_fitness, 2))
print("Average Study:", round(average_study, 2))
print("Average Coding:", round(average_coding, 2))
print("Average Class:", round(average_class, 2))
print("Average Other:", round(average_other, 2))
print("Average Total Tracked:", round(average_total, 2))
print("Average Free Time:", round(average_free, 2))
```

```
Average Sleep: 433.33
Average Fitness: 51.41
Average Study: 237.64
Average Coding: 54.49
Average Class: 205.13
Average Other: 176.92
Average Total Tracked: 1158.92
Average Free Time: 281.08
```

```
def minutes_to_hours(minutes):

    return minutes / 60

print(
    "Average Sleep:",
    round(minutes_to_hours(average_sleep), 2),
    "hours"
)

print(
    "Average Study:",
    round(minutes_to_hours(average_study), 2),
    "hours"
)

print(
    "Average Coding:",
    round(minutes_to_hours(average_coding), 2),
    "hours"
)
```

```
Average Sleep: 7.22 hours
Average Study: 3.96 hours
Average Coding: 0.91 hours
```

```
def calculate_TPI(records):

    return find_average(
        records,
        "coding_minutes"
    )

def calculate_AAI(records):

    if len(records) == 0:
        return 0

    total_academic = 0

    for record in records:
```

```
        academic_time = (
            record["study_minutes"] +
            record["class_minutes"]
        )

        total_academic = (
            total_academic +
            academic_time
        )

    return total_academic / len(records)

def calculate_PhAI(records):

    return find_average(
        records,
        "fitness_minutes"
    )

def calculate_SRI(records):

    return find_average(
        records,
        "sleep_minutes"
    )

def calculate_ABI(records):

    return find_average(
        records,
        "free_minutes"
    )

def calculate_TUI(records):

    return find_average(
        records,
        "total_tracked"
    )

def feeling_to_number(value):
    mapping = {
        "Excellent": 5,
        "Good": 4,
        "Neutral": 3,
        "Low": 2,
        "Stressed": 1
    }

    if value is None:
        return None
```

```
    return mapping.get(str(value).strip().title())


def satisfaction_to_number(value):
    mapping = {
        "Very Satisfied": 5,
        "Satisfied": 4,
        "Neutral": 3,
        "Unsatisfied": 2,
        "Very Unsatisfied": 1
    }

    if value is None:
        return None

    return mapping.get(str(value).strip().title())


def energy_to_number(value):
    mapping = {
        "High": 3,
        "Medium": 2,
        "Low": 1
    }

    if value is None:
        return None

    return mapping.get(str(value).strip().title())

def calculate_EI(records):

    if len(records) == 0:
        return 0

    total_score = 0
    count = 0

    for record in records:

        feeling_score = feeling_to_number(
            record["feeling"]
        )

        satisfaction_score = satisfaction_to_number(
            record["satisfaction"]
        )

        energy_score = energy_to_number(
            record["energy"]
```

```
        )

        if (
            feeling_score is not None
            and satisfaction_score is not None
            and energy_score is not None
        ):

            daily_score = (
                feeling_score
                + satisfaction_score
                + energy_score
            ) / 3

            total_score = total_score + daily_score
            count = count + 1

    if count == 0:
        return 0

    return total_score / count

def calculate_DCI(records, expected_days):
    if expected_days == 0:
        return 0
    return len(records) / expected_days   # then *100 when printing as
%

tpi = calculate_TPI(records)

aai = calculate_AAI(records)

phai = calculate_PhAI(records)

sri = calculate_SRI(records)

abi = calculate_ABI(records)

tui = calculate_TUI(records)

ei = calculate_EI(records)

expected_days = 36

dci = calculate_DCI(
    records,
    expected_days
)
print("TPI :", round(tpi, 2), "minutes/day")

print("AAI :", round(aai, 2), "minutes/day")
```

```r
print("PhAI:", round(phai, 2), "minutes/day")

print("SRI :", round(sri, 2), "minutes/day")

print("ABI :", round(abi, 2), "minutes/day")

print("TUI :", round(tui, 2), "minutes/day")

print("EI  :", round(ei, 2), "/ 5")

print("DCI :", round(dci, 2), "%")
```

```
TPI : 54.49 minutes/day
AAI : 442.77 minutes/day
PhAI: 51.41 minutes/day
SRI : 433.33 minutes/day
ABI : 281.08 minutes/day
TUI : 1158.92 minutes/day
EI  : 2.88 / 5
DCI : 1.08 %
```

```r
def calculate_PAI(
    tpi,
    aai,
    phai,
    sri,
    tui,
    ei,
    dci
):

    pai = (
        (0.15 * tpi) +
        (0.20 * aai) +
        (0.15 * phai) +
        (0.20 * sri) +
        (0.15 * tui) +
        (0.10 * ei) +
        (0.05 * dci)
    )

    return pai

pai = calculate_PAI(
    tpi,
    aai,
    phai,
    sri,
    tui,
    ei,
```

```
    dci
)
print("PAI:", round(pai, 2))
```

```
PAI: 365.29
```

```
def find_correlation(x_values, y_values):

    if len(x_values) != len(y_values):
        return None

    if len(x_values) < 2:
        return None

    x_average = sum(x_values) / len(x_values)

    y_average = sum(y_values) / len(y_values)

    numerator = 0

    x_square_sum = 0

    y_square_sum = 0


    for i in range(len(x_values)):

        x_difference = (
            x_values[i] -
            x_average
        )

        y_difference = (
            y_values[i] -
            y_average
        )


        numerator += (
            x_difference *
            y_difference
        )


        x_square_sum += (
            x_difference ** 2
        )


        y_square_sum += (
            y_difference ** 2
```

```
        )


    denominator = (
        x_square_sum *
        y_square_sum
    ) ** 0.5


    if denominator == 0:
        return None


    return numerator / denominator

def make_matching_lists(
    records,
    first_column,
    second_column
):

    first_values = []
    second_values = []

    for record in records:

        first_value = record[first_column]
        second_value = record[second_column]

        if second_column == "energy":
            second_score = energy_to_number(
                second_value
            )

        elif second_column == "satisfaction":
            second_score = satisfaction_to_number(
                second_value
            )

        elif second_column == "feeling":
            second_score = feeling_to_number(
                second_value
            )

        else:
            second_score = None

        if (
            first_value is not None
            and second_score is not None
        ):
```

```
            first_values.append(
                float(first_value)
            )

            second_values.append(
                float(second_score)
            )

    return first_values, second_values

coding_values, energy_values = make_matching_lists(
    records,
    "coding_minutes",
    "energy"
)

coding_energy = find_correlation(
    coding_values,
    energy_values
)

print(
    "Coding vs Energy:",
    round(coding_energy, 3)
)
```

```
Coding vs Energy: -0.067
```

```
sleep_values, energy_values = make_matching_lists(
    records,
    "sleep_minutes",
    "energy"
)

sleep_energy = find_correlation(
    sleep_values,
    energy_values
)

print(
    "Sleep vs Energy:",
    round(sleep_energy, 3)
)
```

```
Sleep vs Energy: 0.206
```

```
study_values, satisfaction_values = make_matching_lists(
    records,
    "study_minutes",
    "satisfaction"
```

```
)
study_satisfaction = find_correlation(
    study_values,
    satisfaction_values
)
print(
    "Study vs Satisfaction:",
    round(study_satisfaction, 3)
)
```

```
Study vs Satisfaction: 0.089
```

```
def interpret_correlation(value):

    if value is None:
        return "Not enough valid data"

    if value >= 0.70:
        return "Strong positive relationship"

    elif value >= 0.30:
        return "Moderate positive relationship"

    elif value > -0.30:
        return "Weak or little linear relationship"

    elif value > -0.70:
        return "Moderate negative relationship"

    else:
        return "Strong negative relationship"

print(
    "Coding vs Energy:",
    round(coding_energy, 3),
    "->",
    interpret_correlation(coding_energy)
)


print(
    "Sleep vs Energy:",
    round(sleep_energy, 3),
    "->",
    interpret_correlation(sleep_energy)
)


print(
    "Study vs Satisfaction:",
    round(study_satisfaction, 3),
```

```
    "->",
    interpret_correlation(study_satisfaction)
)
```

```
Coding vs Energy: -0.067 -> Weak or little linear relationship
Sleep vs Energy: 0.206 -> Weak or little linear relationship
Study vs Satisfaction: 0.089 -> Weak or little linear relationship
```

```
from datetime import timedelta

def check_date_continuity(records, start_date, end_date):

    if hasattr(start_date, "date"):
        start_date = start_date.date()

    if hasattr(end_date, "date"):
        end_date = end_date.date()

    dates = []

    for record in records:

        record_date = record["date"]

        if hasattr(record_date, "date"):
            record_date = record_date.date()

        dates.append(record_date)

    missing_dates = []

    current_date = start_date

    while current_date <= end_date:

        if current_date not in dates:
            missing_dates.append(current_date)

        current_date = current_date + timedelta(days=1)

    return missing_dates

from datetime import datetime

start_date = datetime(2026, 8, 17)
end_date = datetime(2026, 9, 21)

missing_dates = check_date_continuity(
    records,
    start_date,
    end_date
```

```
)

print("Missing Dates:")

if len(missing_dates) == 0:
    print("No missing dates.")
else:
    for missing_date in missing_dates:
        print(missing_date)
```

```
Missing Dates:
No missing dates.
```

```
def find_duplicate_dates(records):

    dates = []

    duplicates = []
    for record in records:

        current_date = (
            record["date"].date()
        )


        if current_date in dates:

            duplicates.append(
                current_date
            )

        else:

            dates.append(
                current_date
            )


    return duplicates

missing_dates = check_date_continuity(
    records,
    start_date,
    end_date
)

duplicate_dates = find_duplicate_dates(
    records
)

expected_days = (end_date - start_date).days + 1
```

```
print("Expected days:", expected_days)
print("Valid records:", len(records))
print("Invalid records:", len(errors))
print("Missing dates:", len(missing_dates))
print("Duplicate dates:", len(duplicate_dates))

if len(missing_dates) > 0:

    print("\nMissing dates:")

    for missing_date in missing_dates:
        print(missing_date)

if len(duplicate_dates) > 0:

    print("\nDuplicate dates:")

    for duplicate_date in duplicate_dates:
        print(duplicate_date)
```

```
Expected days: 36
Valid records: 39
Invalid records: 0
Missing dates: 0
Duplicate dates: 0
```

```
print(
    "My average coding time was",
    round(average_coding, 2),
    "minutes per day."
)
print(
    "My average study time was",
    round(average_study, 2),
    "minutes per day."
)
print(
    "My average sleep time was",
    round(average_sleep, 2),
    "minutes per day."
)
print(
    "My average fitness time was",
    round(average_fitness, 2),
    "minutes per day."
)
print(
    "My average free/unaccounted time was",
    round(average_free, 2),
```

```
    "minutes per day."
)
print(
    "Coding and Energy correlation:",
    round(coding_energy, 3)
)
print(
    "Sleep and Energy correlation:",
    round(sleep_energy, 3)
)
print(
    "Study and Satisfaction correlation:",
    round(study_satisfaction, 3)
)
```

```
My average coding time was 54.49 minutes per day.
My average study time was 237.64 minutes per day.
My average sleep time was 433.33 minutes per day.
My average fitness time was 51.41 minutes per day.
My average free/unaccounted time was 281.08 minutes per day.
Coding and Energy correlation: -0.067
Sleep and Energy correlation: 0.206
Study and Satisfaction correlation: 0.089
```